

Manifold Collapse: A 32-Byte Identity for a Versioned Deterministic Runtime

A conceptual systems architecture and falsifiable research program Leon Calvin Long II · SquirrelOS Technologies LLC · September 23, 2026 · Revised public edition v1.1 ORCID: 0009-0002-1140-9568 · DOI: 10.5281/zenodo.22921230 · License: CC BY 4.0

Status and provenance. This is a conceptual proposal, not a proof of a lossless seed-only reconstruction of an arbitrary application, an independently replicated benchmark, or a claim of production readiness. The author's earlier 36-page draft, **Manifold Collapse: Encoding a No-Code Builder as a 32-Byte Deterministic Runtime Identity with Semantic Fiber Preservation** (September 23, 2026), supplied the organizing idea. This revised public edition corrects information-theoretic and cryptographic overclaims, removes unverified case-study figures, and states testable acceptance criteria. The earlier draft is not a receipted experiment and is not included as supporting evidence. AI assistance was used in editing this revision; the author retains responsibility for technical review. Publication is not a representation that any particular claim is covered by a particular patent application.

Abstract

Can a fixed-size identity identify and reproducibly reconstitute a complex interactive application? A 32-byte token can commit to a versioned runtime **recipe** or select an instance from a bounded generative family, provided the reconstruction machinery, immutable schema, assets, model versions, and any needed state are available and authenticated. The token cannot, by itself, carry arbitrary application state or invert a cryptographic hash. We present a conditional architecture that separates a compact identity, a shared substrate, and a deterministic reconstruction procedure. We use the language of base structure and semantic fibers as a design vocabulary, without assuming that the discrete configuration space is a smooth manifold or that an LLM interface emerges automatically. We specify observable equivalence at a bounded interface, state necessary information-capacity conditions, and propose reproducibility, integrity, ablation, and adversarial tests. This paper connects the author's 32-byte blueprint-pointer idea to the **Deterministic Runtime Recipe Book** and related runtime research while keeping hypotheses separate from measurements.

Keywords: deterministic reconstruction; content-addressed identity; versioned runtime; semantic contracts; information bounds; reproducible builds; application blueprints.

1. The role of the 32-byte anchor

The central proposal is to treat a 256-bit token as an **identity and entry point**, not as a miniature copy of a running application. A useful analogy is a commit identifier paired with a trusted repository and a deterministic build recipe: the identifier is small, but the reconstructable object also depends on code and content held elsewhere. The original insight is architectural: constrain and version the reconstruction environment until a small identity is sufficient to select a particular admissible behavior. It is not a claim that 32 bytes literally contain unlimited data.

Let Γ denote a specified reconstruction substrate: a signed and versioned schema registry, canonicalization rules, executable decoder, content-addressed assets, relevant model weights and configuration, and a policy for external inputs. Let $s \in \{0,1\}^{256}$ be the identity. We propose a reconstruction interface

$R_\Gamma(s, i) \rightarrow (\text{runtime state, verification receipt}) \mid \text{error},$

where i is explicitly declared initialization data. If Γ and i are immutable and fixed, and R is deterministic under a defined execution model, repeated valid calls can be tested for equal output. If Γ , i , the model, clock, network, or random stream changes, the same seed alone is **not** a complete state description.

The **Recipe Book** supplies the engineering layer: invariant boundaries, blueprint validation, a kernel truth chain, and binding rules. The present paper supplies the identity and reconstruction research question. Related manifold papers supply a geometric interpretation to investigate, not an automatic theorem of software correctness. These are complementary roles, not interchangeable kinds of evidence.

2. What 256 bits can and cannot guarantee

For a fixed Γ and i , let S be the finite set of allowed runtime states and $\approx$ a clearly defined observational-equivalence relation. If every equivalence class is to receive a distinct 256-bit code that permits exact class reconstruction, a **necessary condition** is $|S/\approx| \leq 2^{256}$. For a finite, effectively enumerable $S/\approx$ with an agreed canonical decoder and enough time and storage in Γ , this is also a theoretical sufficiency condition for an index code. It does not make decoding fast or eliminate the cost of the shared substrate.

A small *average* Shannon entropy does not imply that every possible state fits losslessly in 256 fixed bits. Nor does correlation alone establish a 256-bit worst-case bound. One must define the admissible distribution or finite family and demonstrate the bound for that family. As a simple counterexample, 47 independently editable labels drawn from 65,536 possibilities already span $47 \times 16 = 752$ bits of possible choices before edge weights, bindings, or chat history. If only a restricted, reproducibly generated subset is valid, define it and show the generator.

A cryptographic digest commits to data that must be available separately; it is not a general-purpose decompressor. In particular, a 64-bit aggregate or Merkle root does not contain a reversible encoding of thousands of independent configuration bits. A digest match can verify a candidate supplied by a registry; it cannot construct the unique preimage from the digest alone. Truncating a hash to 64 bits also creates collision risk that must be budgeted for the number of stored objects. A CRC-32C is an accidental-corruption check, not an authenticity mechanism. The string s is an *identity* if it verifies and resolves correctly under a trusted Γ , not proof that all information was embedded in it.

This preserves the generative-seed distinction: a fixed recipe can expand a compact parameter vector into a much larger output, but only across the family that the recipe is capable of generating. Both the recipe and any external assets count toward the full reconstruction contract.

3. A testable reconstruction architecture

A prototype can implement the following pipeline:

- **Freeze and publish a manifest.** Record schema/version IDs, decoder source and build hash, asset digests, model identifier, canonicalization rules, environment assumptions, and the boundary of permitted I/O. Sign the manifest and preserve earlier versions.
- **Canonicalize and identify.** Canonically encode a blueprint and derive a cryptographic commitment. If a 32-byte identity is used, specify whether it is a full content digest, a generated parameter vector, or a structured token. These are different designs. Avoid treating arbitrary slices of one digest as independently invertible fields.
- **Resolve dependencies.** Look up the manifest and, where required, load code and content from a local verified registry. A local immutable registry can make *the reconstruction computation* hermetic, but the

registry is still an external information source relative to the token.

- **Reconstruct and validate.** Instantiate the graph and bindings; enforce invariant contracts and version checks. Before exposing the runtime, compare canonical output digests and test declared semantic invariants. On mismatch or unavailable dependencies, return a structured error, not a guessed reconstruction.
- **Issue a receipt.** Record input identity, manifest and dependency digests, environment, resulting runtime digest, observation-test version, timing methodology, and failure modes. Exclude credentials and private/customer data.

An example pseudocode contract is:

```
rehydrate(identity, manifest, registry, initialization):
    verify_signature(manifest)
    verify_version_and_dependencies(identity, manifest, registry)
    blueprint = resolve_or_generate(identity, manifest, registry)
    runtime = deterministic_assemble(blueprint, initialization)
    assert validate_invariants(runtime, manifest)
    assert canonical_digest(runtime) == manifest.expected_runtime_digest(identity)
    return runtime, reproducibility_receipt(identity, manifest, runtime)
```

The resolver may generate from explicit parameters or retrieve committed material. Neither mechanism permits an inverse cryptographic hash. Pure functional behavior is a property to establish for the bounded assembly operation, not for an unrestricted chat service that may call an LLM or external APIs.

4. Semantic fibers as a design model

The original manuscript describes an application skeleton as a base and per-node roles, intent labels, relationships, and contracts as fibers. That vocabulary is useful for separating **structure** from **behavioral meaning**. A practical semantic fiber might contain a versioned role identifier, a canonical intent label, a binding target, and executable contract tests. A canonical hash of that bundle can authenticate it *after the bundle has been retrieved*, not recover it from the hash.

A no-code builder's graph and discrete labels do not automatically form a smooth differentiable manifold. To use manifold and fiber-bundle theorems literally, one must define the underlying spaces, local charts, transition maps, topology, smoothness assumptions, and the relationship of points to reachable program states. Otherwise, geometric terms in this paper are modeling hypotheses. A global section is a map selecting a value in each fiber; it is not itself a 32-byte bitstring. The existence of such a section is not implied by an entropy estimate or local triviality. Discrete stratified configuration spaces or typed graph models may be more appropriate for an implementation.

Likewise, a versioned LLM interface can be *bound* to the reconstructed runtime through explicit intent schemas, prompts/tools, model versions, and evaluation tests. A 64-bit key does not automatically encode a high-dimensional alignment matrix or imply a smooth, invertible map into a model's latent space. The question whether a reusable projection mechanism works across applications is an empirical research question.

5. Equivalence and determinism claims

Define a finite, published test domain T : UI snapshots at specified viewport sizes, graph-edit operations, schema binding checks, deterministic tool responses, and selected chat-routing inputs with the LLM model and sampling configuration pinned. Two instances pass **T-equivalence** when their canonical observations match for every test in T . This is a useful regression criterion but weaker than equivalence under *all possible* interaction sequences.

Full behavioral equivalence requires a mathematical model of the allowed transitions and a proof, or a carefully bounded state space that can be exhaustively explored. A 120/120 routing match, if reproduced and receipted, would be evidence only on those 120 inputs, not a universal guarantee. Generated chat text may remain nondeterministic despite deterministic routing unless the complete inference environment is controlled. Security, latency and fidelity require distinct measurements.

6. Falsifiable experiment and reporting protocol

The following protocol is proposed, **not reported as already executed**:

- Publish an implementation, canonical encoder/decoder specification, test harness, sample non-sensitive blueprint, and hashes before the evaluation run. Identify all inputs stored outside the 32-byte token.
- Freeze the implementation and measurement plan. On a clean second machine, resolve Γ and reconstruct from the identity; then compare canonical graph, bindings, assets and contract results against a baseline.
- Repeat for multiple seeds, independently generated configurations and schema versions. Probe intentional collisions, unavailable assets, mismatched schema versions, changed model weights, malicious registry content, and corrupted identities.
- Report wall-clock timing with hardware, warm/cold cache status, sample count, distributions and confidence intervals; separate fetch/setup time from local assembly time.
- Run ablations: token only without Γ ; token plus schema; token plus full verified registry. This exposes where the information actually resides.
- Release anonymized receipts and failing traces. A single successful demo supports feasibility for that configuration, not an information-theoretic theorem for arbitrary builders.

Acceptance criteria: (a) declared reproducibility of outputs on matched dependencies; (b) verified failure when required dependencies are missing or altered; (c) no unexplained hidden state or undeclared I/O; (d) successful invariant and regression tests on a published test domain; and (e) measurement claims traceable to raw logs. A failure is a research finding and must not be erased from the version history.

The earlier draft cited a 47-node builder, a 32-byte hexadecimal specimen, timings below 12 ms, 55 matching semantic hashes and 120 chat tests. No primary machine-readable logs, implementation, registry or test harness accompanies the supplied draft. Accordingly **none of those are presented here as verified outcomes**. The hexadecimal specimen is omitted rather than offered as a reproducible artifact. The alleged 211-bit entropy estimate cannot establish a worst-case bound without the underlying state model and measurement procedure. This is a disclosure of the evidentiary gap, not a claim that those experiments did or did not occur.

7. Relationship to the wider program and limitations

This paper is a conceptual bridge between the author's early pointer/blueprint insight, the *Deterministic Runtime Recipe Book* engineering specification, and a reproducible runtime research program. It does not incorporate those works' unverified mathematical claims by reference, certify a no-code application builder, or say that a later stress-test benchmark validates this 32-byte reconstruction mechanism. A chaos run exercising healing gates and an identity-rehydration experiment are different tests and require separate receipts.

Open problems include a precise equivalence relation for semantic UI state; complete versioning of registry, model, assets and decoder; collision and adversarial risk under a 64-bit truncated lookup; testable geometry for discrete software states; bounded and explicit conversational history; and proofs or experiments for efficient reconstruction. A 256-bit digest can be an excellent stable handle if its dependency graph is preserved. It cannot be the entire dependency graph.

8. Conclusion

The 32-byte identity proposal is useful as a disciplined way to ask *which information must be stored where* for reproducible runtime recovery. Its strongest defensible form is conditional: a small token, an authenticated and versioned shared substrate, a specified reconstruction algorithm, and published equivalence tests can make a bounded runtime family reconstitutable. The path from idea to result is an implementation and receipt trail, not an inverse hash or an entropy assertion. This revised edition preserves the originating vision while making the falsifiers and information boundary visible.

Related works and source context

- Leon Calvin Long II, *Deterministic Runtime Recipe Book*, version 1.0.0, September 2026. Companion engineering specification supplied by the author; document status and any separate public record should be verified independently.
- Leon Calvin Long II, *Manifold as Deterministic Runtime: How Mathematical Structure Implies Execution Semantics*, DOI: [10.5281/zenodo.22881459](https://doi.org/10.5281/zenodo.22881459). Its geometric thesis is a related proposal, not a proof of the architecture described here.
- Leon Calvin Long II, *Aurora Runtime: A Deterministic Execution Model for Manifold-Coherent Distributed Systems*, DOI: [10.5281/zenodo.22878201](https://doi.org/10.5281/zenodo.22878201).
- J. Ziv and A. Lempel, "Compression of Individual Sequences via Variable-Rate Coding," *IEEE Transactions on Information Theory* 24(5), 1978. Lossless compression does not furnish a universal inverse to a digest.
- The 256-bit cardinality condition is the elementary pigeonhole principle; no new information-theoretic theorem is claimed.

AI-use, scope, and prototype disclosure

The supplied earlier draft contains statements and empirical-looking examples without accompanying receipts. This edition was edited with AI assistance to distinguish proposal, conditions and verified results; it does not attribute the original draft to any particular AI product. All results in this edition are mathematical conditions or proposed experiments unless explicitly labeled as a previously published companion record. No current customer, tenant, credential, wallet or protected health information is included. Citation to pending patents or a company portfolio does not establish patent coverage of this particular embodiment. No production-readiness or performance warranty is implied.

This software is a prototype and is provided for educational and research purposes only. It is not intended for production use, commercial deployment, or safety-critical environments. All systems are experimental and may contain defects on them.